Deep Generative Models

OVERVIEW

Roadmap for this session:

- Why learn to **generate** data, and what it is used for
- **Variational Autoencoders (VAEs)** — latent variables and the ELBO
- **Diffusion models (DDPM)** — generating by learning to denoise
- Case study: **Imagen**, Google's text-to-image diffusion model

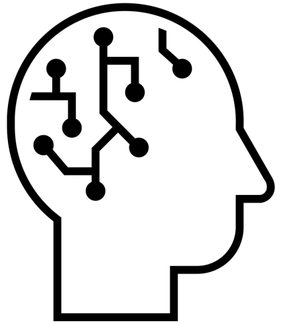
Deep Generative Models

Learning to generate data

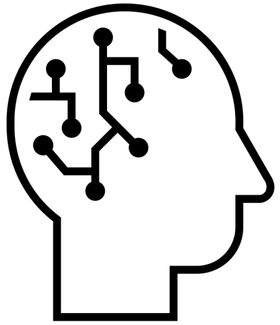


Samples from a Data Distribution

Train



Neural Network



Sample



EXPLANATION

Goal: learn the probability distribution p(x) that the training images came from.

Train: show the network many examples (no labels needed — the data is its own supervision).

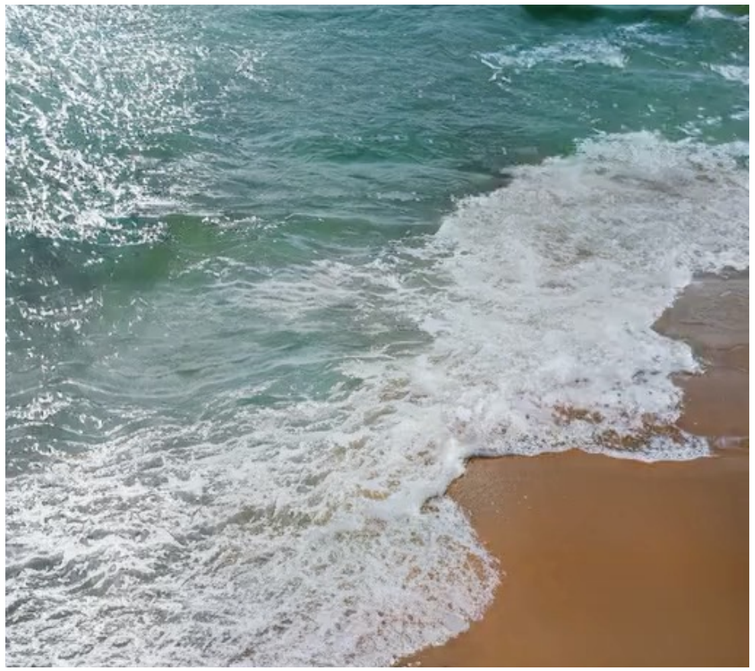
Sample: draw brand-new x from the learned p(x). The result should look like the data, but not be a copy of any training image.

Application (1): Content Generation

StyleGAN3 example images







EXPLANATION

None of these three images is real. A **GAN generator** maps a random noise vector z to an image, and is trained against a discriminator that tries to tell real from fake. StyleGAN3's contribution is making the generator **alias-free**: fine details (hair, wrinkles, wave foam) move smoothly with the object instead of sticking to fixed pixel positions — important for video and animation.

Karras et al. Alias-Free Generative Adversarial Networks, NeurIPS 2021

Application (2): Representation Learning

Learning from limited labels





EXPLANATION

A generator that can draw cars and faces must already “know” their parts (wheels, doors, eyes, hair). **DatasetGAN** exploits this: annotate only a few dozen generated images, train a small decoder on the generator's internal features, then produce an unlimited labeled dataset (image + segmentation mask) for free.

Zhang et al., DatasetGAN: Efficient Labeled Data Factory with Minimal Human Effort, CVPR 2021

Li et al., Semantic Segmentation with Generative Models: Semi-Supervised Learning and Strong Out-of-Domain Generalization, CVPR 2021

Application (3): Artistic Tools

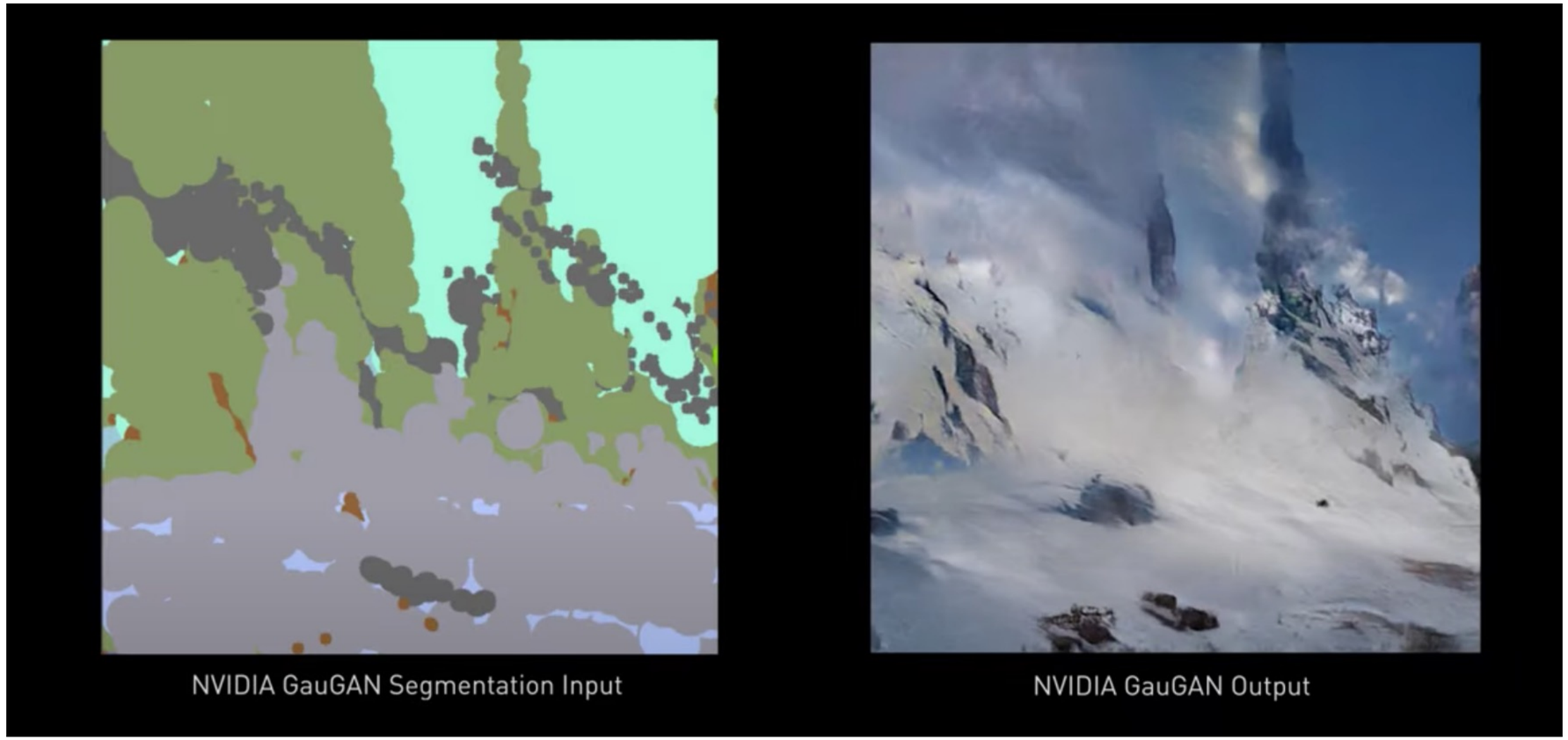
NVIDIA GauGAN

EXPLANATION

Input: a semantic map — each colour is a class (sky, mountain, snow, rock).

Output: a photorealistic image that respects that layout.

An artist edits the coarse map; the model fills in realistic texture and lighting.



EXPLANATION

Key idea (SPADE): ordinary normalization layers “wash out” the input map.

SPADE instead uses the map to predict a per-pixel scale and shift after every normalization, so the layout steers the generator at every layer.

Park et al., Semantic Image Synthesis with Spatially-Adaptive Normalization, CVPR 2019

2022 / 2023 : The year of generative modeling?







Parti

Pathways Autoregressive Text-to-Image Model

DALL·E 2

Stable Diffusion

Imagen, **Imagen Video**, **Parti**, **Phenaki** (Google) · **DALL·E 2** (OpenAI) · **Stable Diffusion** (open weights). Most are **diffusion** models; Parti and Phenaki are token-based exceptions.

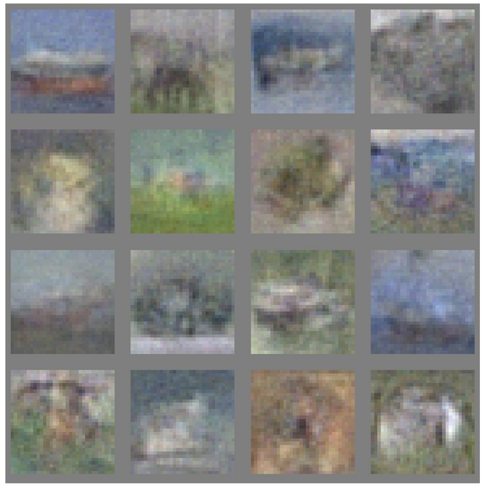
Where we came from

VAEs, 2013



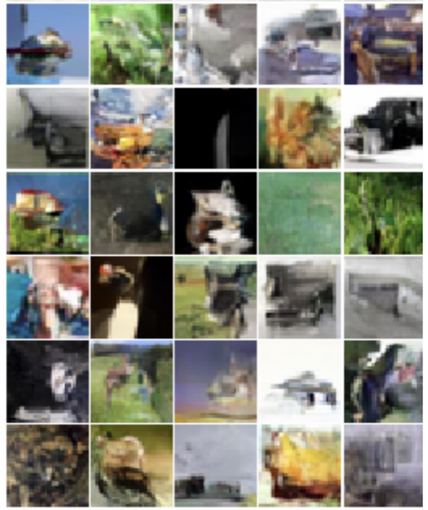
VAE — encoder + decoder trained with a likelihood bound. Stable, but samples are blurry.

GANs, 2014



GAN — generator vs. discriminator game. Sharp results, but training is unstable (mode collapse).

PixelCNN, 2016



PixelCNN — autoregressive: one pixel at a time. Exact likelihood, very slow to sample.

BigGAN, 2019



BigGAN — GANs at scale (large batches, class conditioning) reach ImageNet-level realism.

Imagen, 2022



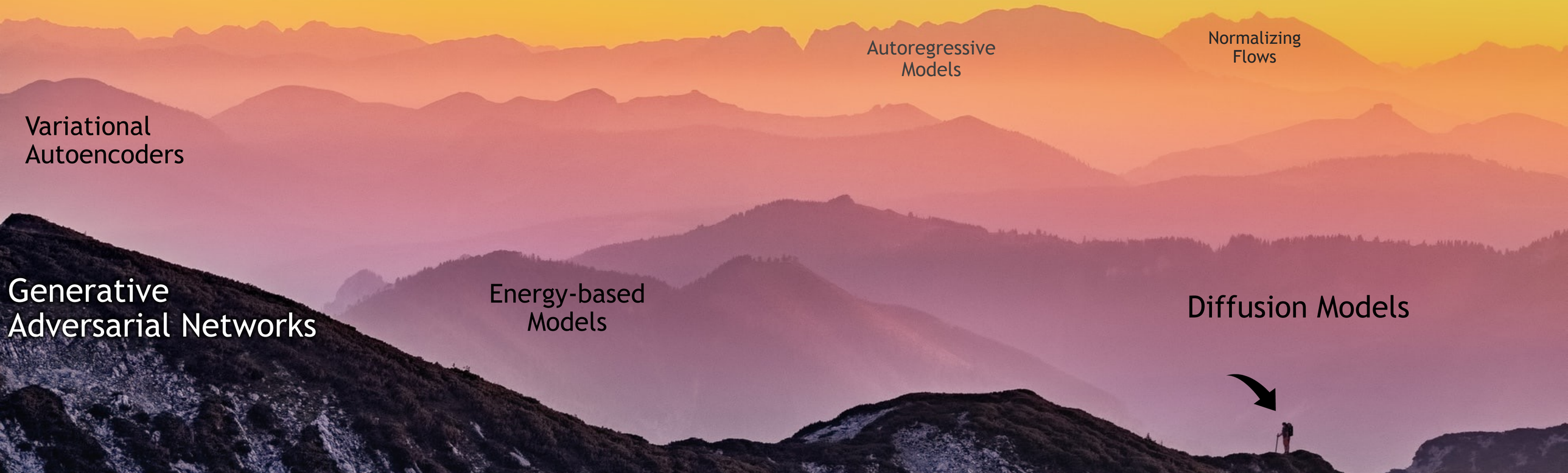
Imagen — text-conditioned diffusion: photorealism plus control through language.

The Landscape of Deep Generative Models

READING THE LANDSCAPE

Six families, one goal (model p(x)), different trade-offs:

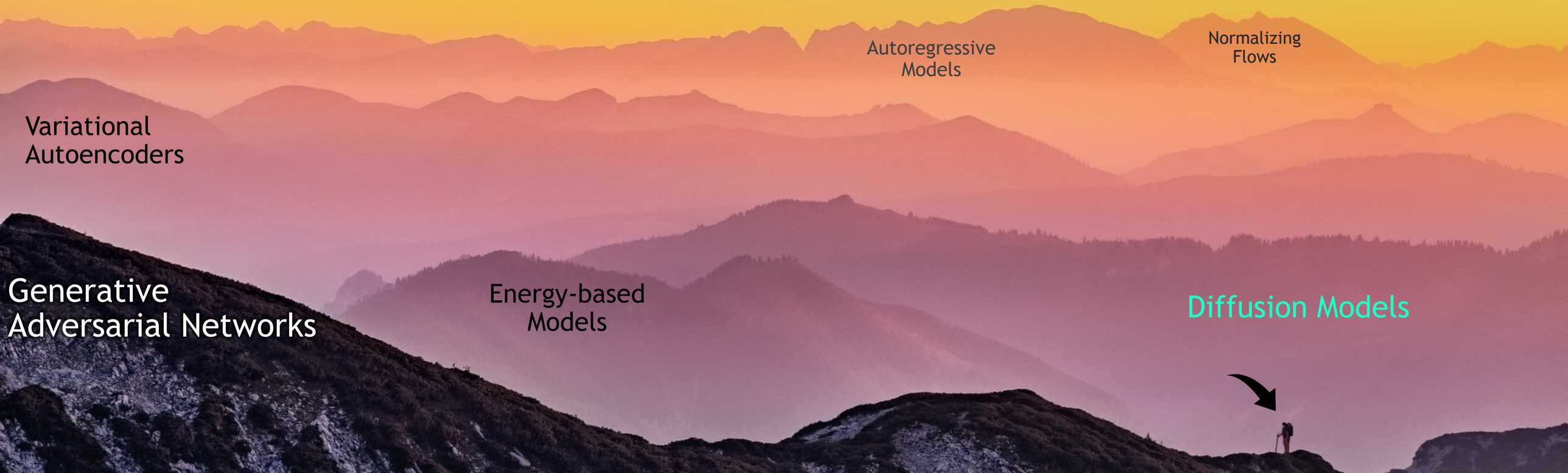
- **GANs** — sharp & fast, unstable to train; **VAEs** — stable & fast, often blurry
- **Autoregressive** — exact likelihood, slow sequential sampling; **Normalizing flows** — exact likelihood, restricted (invertible) architectures
- **Energy-based** — very flexible, costly MCMC sampling; **Diffusion** — high quality & stable, but many sampling steps



The Landscape of Deep Generative Models

HIGHLIGHT

Why diffusion is the focus: it combines the stability of likelihood-based training (like VAEs) with sample quality that rivals or beats GANs, and it scales well with data and compute. The rest of this session builds it up from VAEs, because a diffusion model is mathematically a special kind of hierarchical VAE.



AI Art

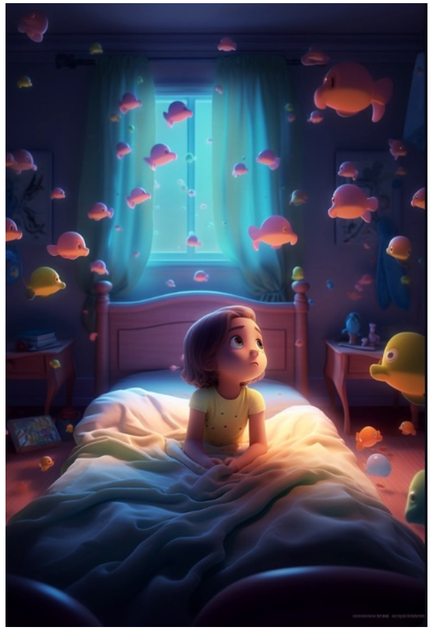












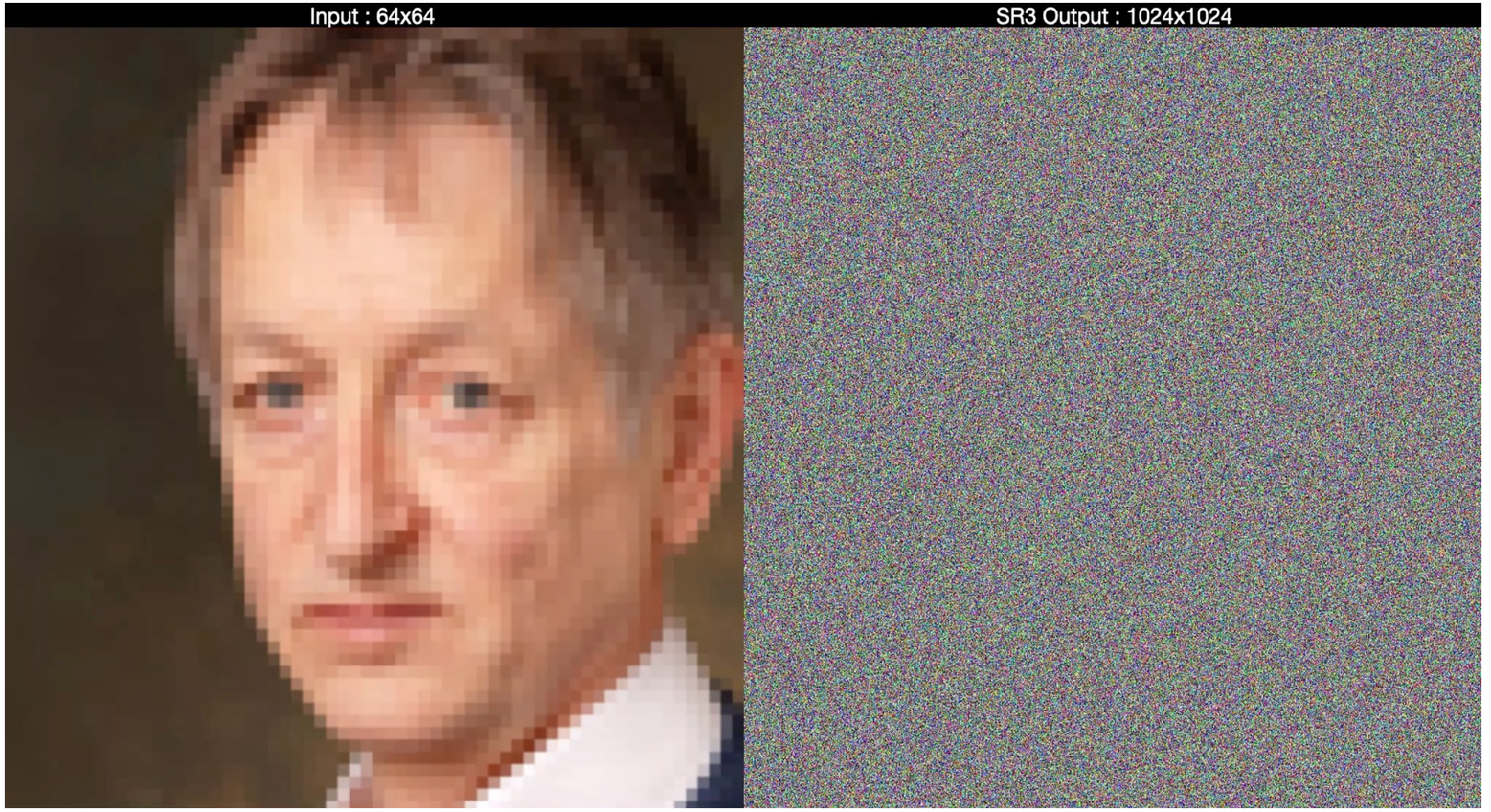


EXPLANATION

Every image here was produced by a diffusion-based model (Midjourney v5) from a **text prompt alone** — spanning illustration, sci-fi concept art, macro photography and product shots.

Midjourney v5

Applications: Super-resolution



EXPLANATION

Task: 64×64 → 1024×1024 (16× upscaling).

The right panel is pure noise because it shows the **starting point** of sampling (step T). The model repeatedly denoises it, always **conditioned on the low-res input**, until a sharp face appears.

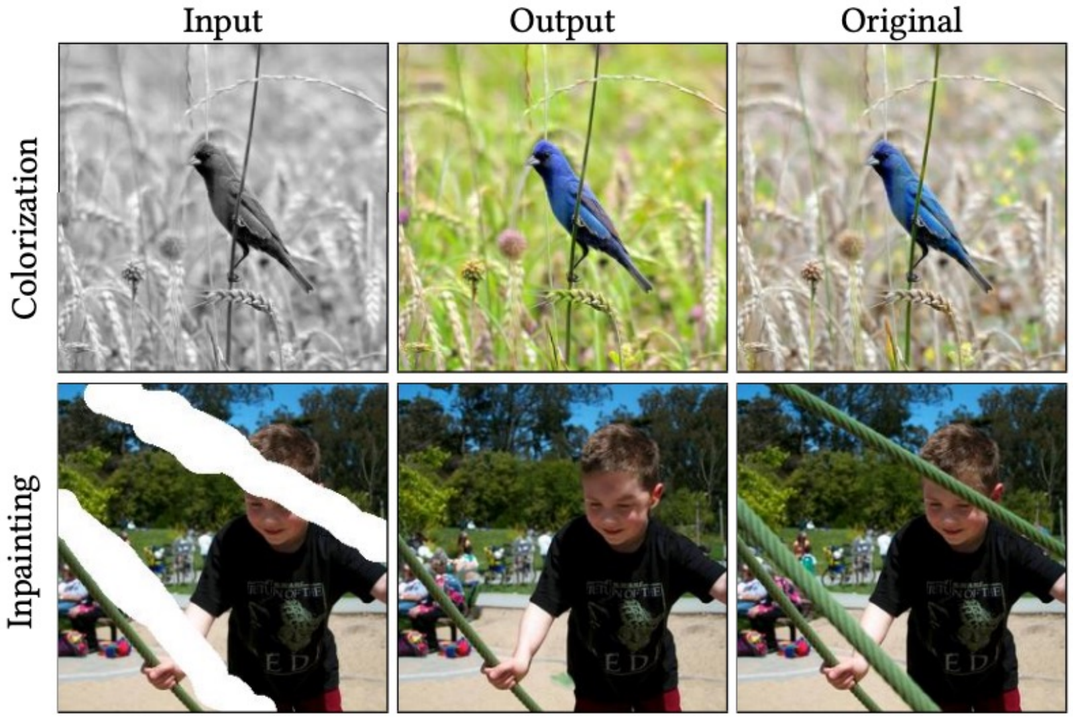
Unlike a regression model that averages possibilities (blur), diffusion **samples one plausible** high-res detail set.

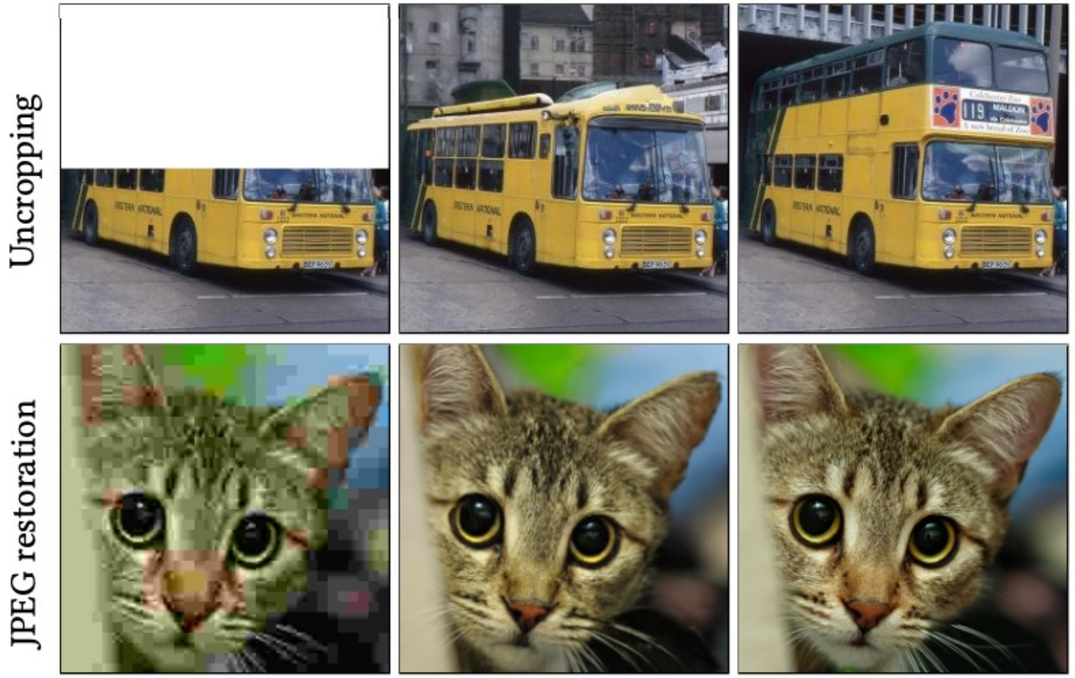
Palette: Image-to-Image Diffusion Models, 2022

Applications: Colorization, Inpainting, Restoration

EXPLANATION

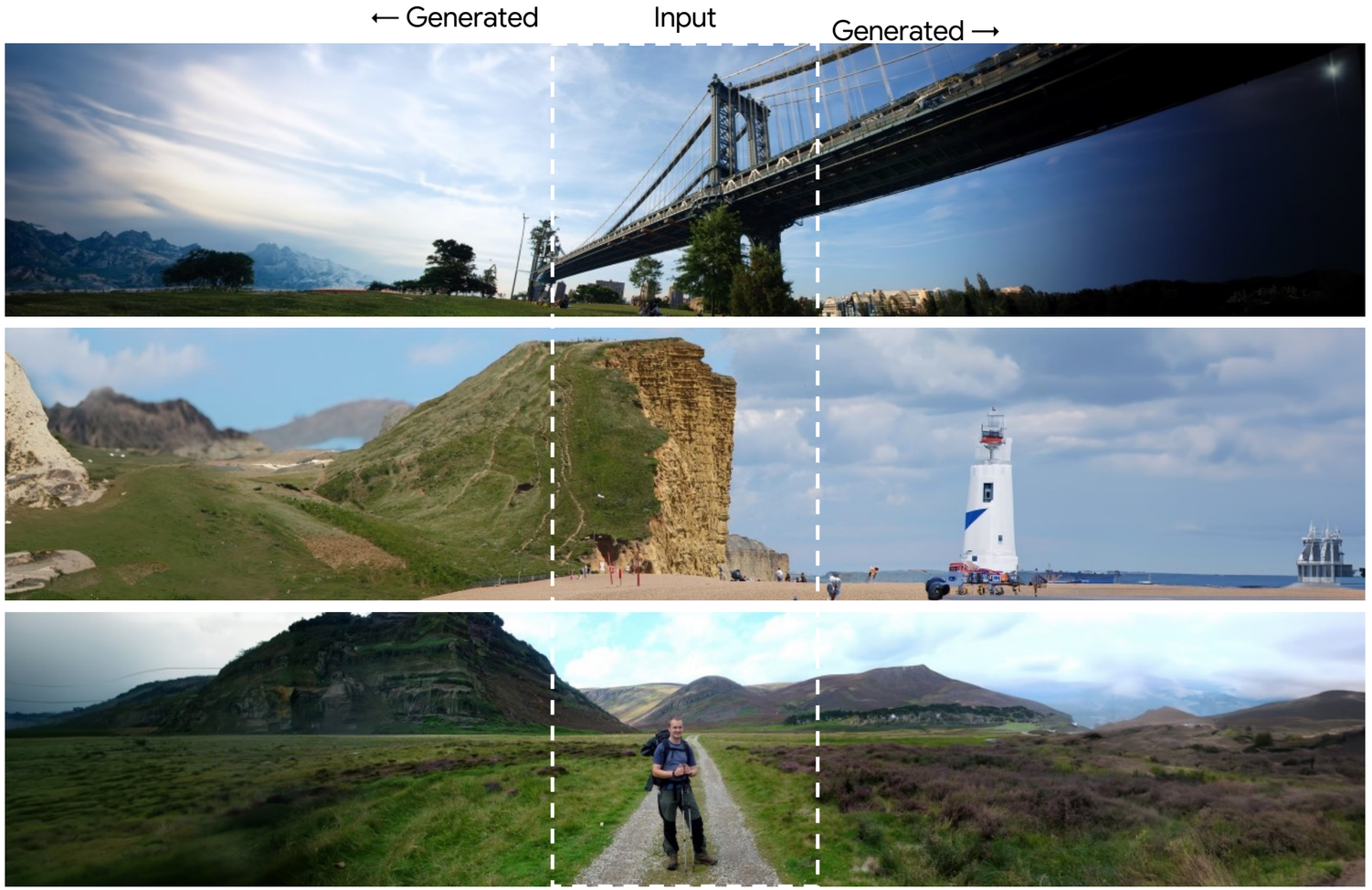
Palette treats all of these as one problem: generate y given an input image x. The network and the loss are **identical** for every task — only the (input, target) training pairs change. The conditioning image is simply concatenated to the noisy image at every denoising step. One multi-task model can even handle several tasks at once.





Palette: Image-to-Image Diffusion Models, 2022

Applications: Outfilling



EXPLANATION

Outfilling (uncropping): only the centre strip is real; everything outside the dashed lines is invented.

The model must stay consistent with the input — matching horizon, lighting, perspective and style — while hallucinating plausible new content.

Applied repeatedly, it can extend a photo into a panorama.

Palette: Image-to-Image Diffusion Models, 2022

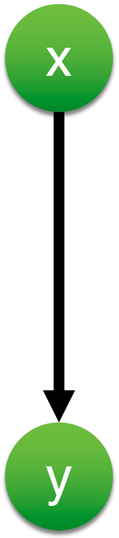
Variational Autoencoders

OVERVIEW

The first family we study in detail. Key ideas: explain each image with hidden **latent variables** z, learn an **encoder** q(z|x) and a **decoder** p(x|z), and train both with the **evidence lower bound (ELBO)**. These ideas carry over directly to diffusion models.

Classifying chocolate





$$p(\mathbf{y}|\mathbf{x}) = \text{Categorical}(\mathbf{y}; \mathbf{f}(\mathbf{x}))$$

“chocolate”

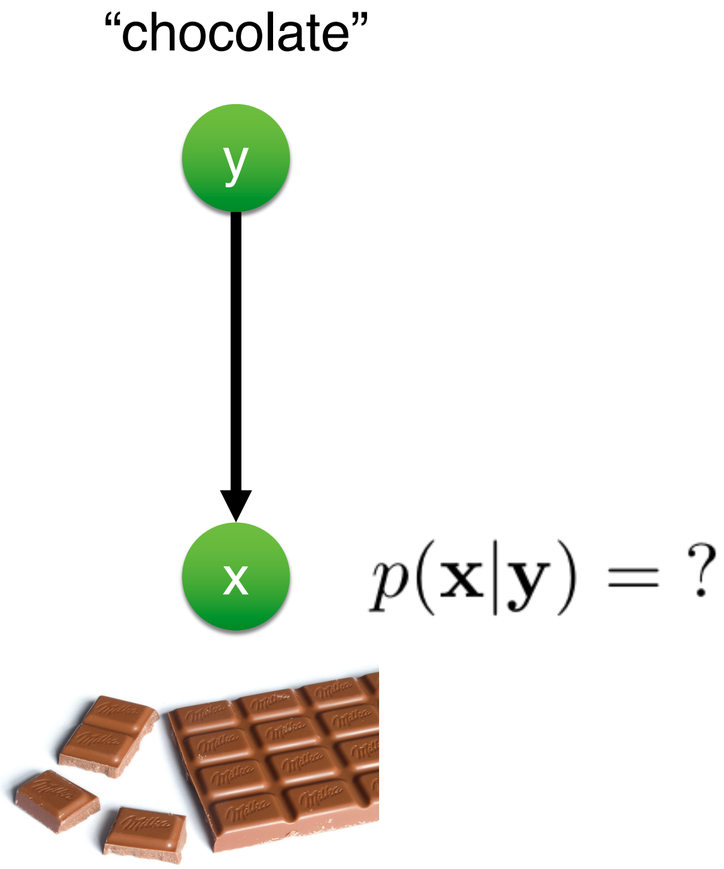
EXPLANATION

This is ordinary **discriminative** learning: given an image x, predict a label y.

The output is small (one of K classes), so a softmax over f(x) is enough to express p(y|x) completely.

Uncertainty here is only about **which label** — a few numbers.

Generating chocolate



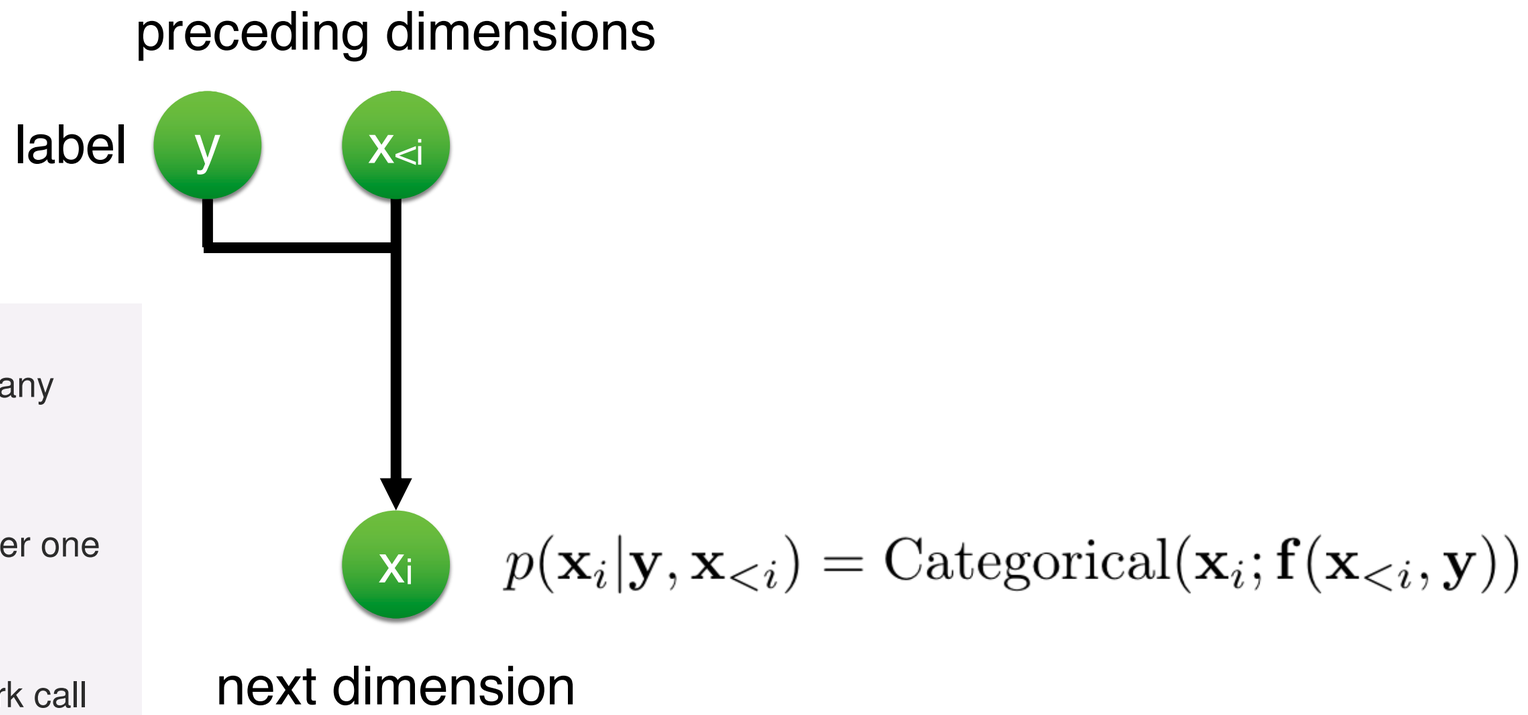
EXPLANATION

Now reverse the arrow: given “chocolate”, produce an image.

x has **~200,000 numbers** (256×256×3) that are strongly correlated. Predicting each pixel independently gives the **average** chocolate — a blurry blob.

We need a way to model the **dependencies**. Two classic strategies follow: **autoregressive** and **latent variable**.

Autoregressive approach



EXPLANATION

Idea: break the hard problem into many easy ones using the chain rule:

$p(x|y) = \prod_i p(x_i \mid y, x_{<i})$

Each factor is just a classification over one pixel's value (e.g. 256 levels).

+ exact likelihood, very expressive

– sampling is sequential: one network call per dimension (PixelCNN, and GPT for text).

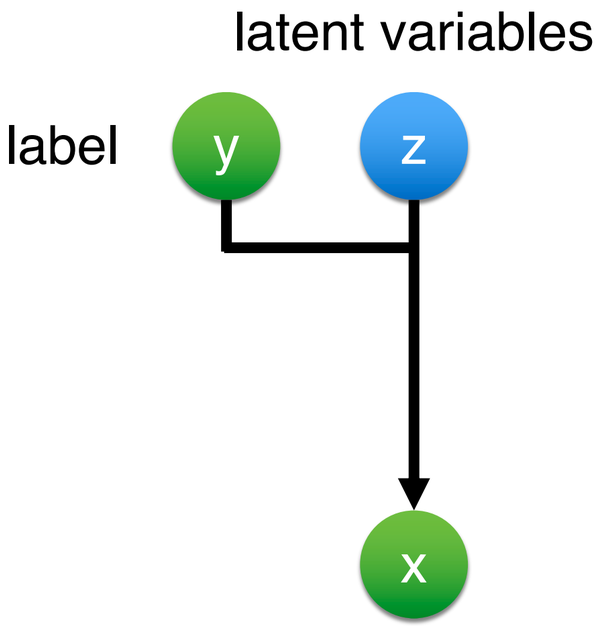
Latent variable approach

EXPLANATION

z = hidden factors not given by the label (shape, angle, lighting, how broken the bar is).

Prior p(z) is a simple Gaussian, and p(x|y,z) is a simple Gaussian too — but because its mean is a deep network of z, integrating over z gives an **infinite mixture** of Gaussians: a very flexible p(x|y).

Sampling is one shot: draw z, run the network once.





$$
\begin{aligned}
p(\mathbf{z}) &= \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \\
p_\theta(\mathbf{x}|\mathbf{y}, \mathbf{z}) &= \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_\theta(\mathbf{z}, \mathbf{y}), \sigma^2 \mathbf{I}) \\
p_\theta(\mathbf{x}|\mathbf{y}) &= \int_{\mathbf{z}} p(\mathbf{x}|\mathbf{y}, \mathbf{z}) p(\mathbf{z})\, d\mathbf{z} \\
&= \text{flexible distribution}
\end{aligned}
$$

EXPLANATION

vs. autoregressive: all pixels are generated in parallel; the correlations between them come from sharing the same z.

Latent variable approach - without y

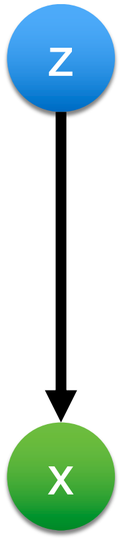
EXPLANATION

Same model, but **unconditional**: no label at all.

Now z must capture **everything** about the image — including what object it is — learned purely from data, without supervision.

To generate: sample z ~ N(0, I) and decode with $\mu_\theta(z)$.

latent variables



full image



$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$$

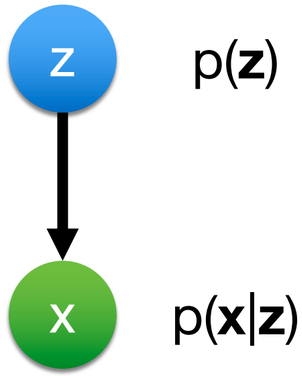
$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_\theta(\mathbf{z}), \sigma^2\mathbf{I})$$

$$p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})\, d\mathbf{z}$$

$$= \text{flexible distribution}$$

Optimization

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$$
$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_\theta(\mathbf{z}), \sigma^2\mathbf{I})$$
$$p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})\, d\mathbf{z}$$
$$= \text{flexible distribution}$$



Generative model
p(x,z)

- Problem:
  - Marginal likelihood p(x) is intractable
  - So can’t do maximum likelihood directly

EXPLANATION

Why is p(x) intractable? The integral is over a high-dimensional z, with a neural network inside — no closed form. Estimating it by sampling z from the prior almost never works: for a given image x, nearly every random z decodes to something unrelated, so the estimate has huge variance. We would need to know **which z are likely for this x** — the posterior p(z|x) — which is also intractable. The fix on the next slide: **learn an approximation** to it.

Variational Autoencoders (VAEs)

- We introduce an **inference model** q(z|x)

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \boldsymbol{\Sigma}_\phi(\mathbf{x}))$$

- This allows us to efficiently optimize the log-likelihood, through the **evidence lower bound** (ELBO).

$$\log p_{\theta,\phi}(\mathbf{x}) \geq \mathrm{ELBO}(\mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x},\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]$$

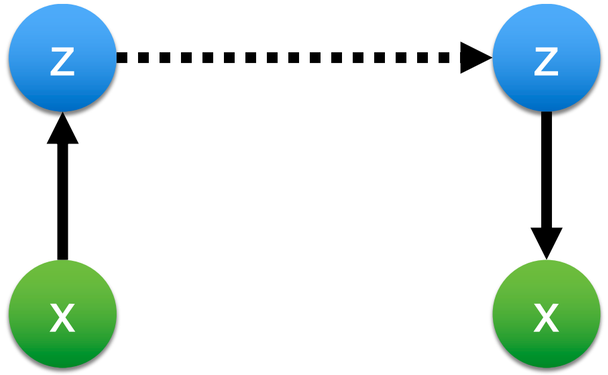
- We optimize q(z|x) and p(x,z) jointly w.r.t. ELBO

- Bound is tight with the right q(z|x)

EXPLANATION

Encoder q(z|x): image → mean and variance of z. **Decoder** p(x|z): z → image.

ELBO = **reconstruction** term E_q[log p(x|z)] – **KL**(q(z|x) ‖ p(z)). Reconstruct well, while keeping codes close to the N(0, I) prior.



Inference model **q(z|x)**

Generative model **p(x,z)**

EXPLANATION

Reparameterization trick: write z = μ + σ ⊙ ε with ε ~ N(0, I), so gradients flow through the sampling step and both networks train with ordinary backprop.

The gap between log p(x) and the ELBO is exactly KL(q(z|x) ‖ p(z|x)).

[Kingma and Welling, 2013]

Hierarchical VAEs

- “Flat” VAEs suffer from simple priors
- Making both inference model and generative model hierarchical

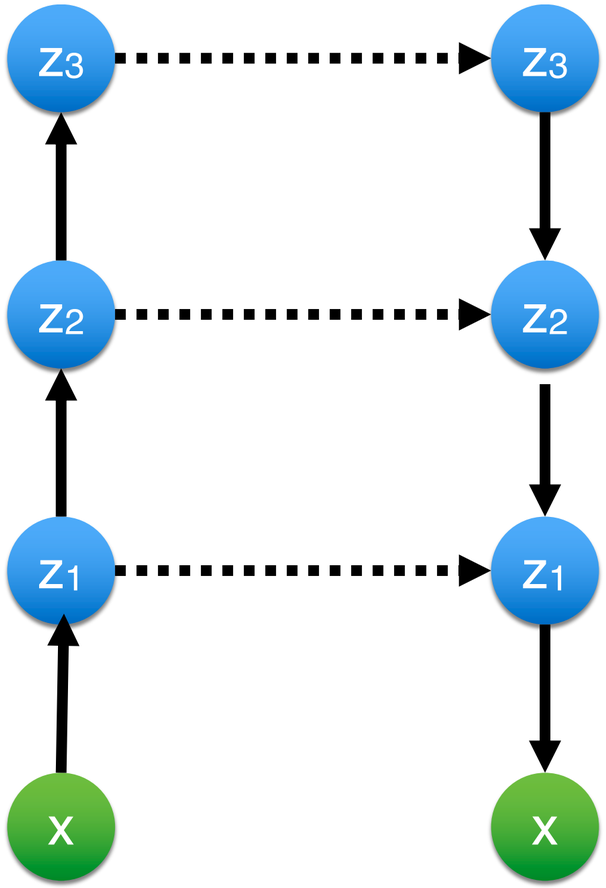
$$q_\phi(\mathbf{z}_{1,2,3}|\mathbf{x}) = q_\phi(\mathbf{z}_1|\mathbf{x})q_\phi(\mathbf{z}_2|\mathbf{z}_1)q_\phi(\mathbf{z}_3|\mathbf{z}_2)$$

$$p_\theta(\mathbf{z}_{1,2,3}) = p_\theta(\mathbf{z}_3)p_\theta(\mathbf{z}_2|\mathbf{z}_3)p_\theta(\mathbf{z}_1|\mathbf{z}_2)p_\theta(\mathbf{x}|\mathbf{z}_1)$$

- Better likelihoods are achieved with hierarchies of latent variables

EXPLANATION

A single Gaussian prior is too simple to describe all the structure in images. Stacking latents lets each level model a different scale: $\mathbf{z_3}$ global layout → $\mathbf{z_2}$ object parts → $\mathbf{z_1}$ fine texture. Examples: Ladder VAE, NVAE. Keep this picture in mind — a diffusion model has exactly this chain structure.



Inference model
q(z|x)

Generative model
p(x,z)

[Kingma and Welling, 2017]

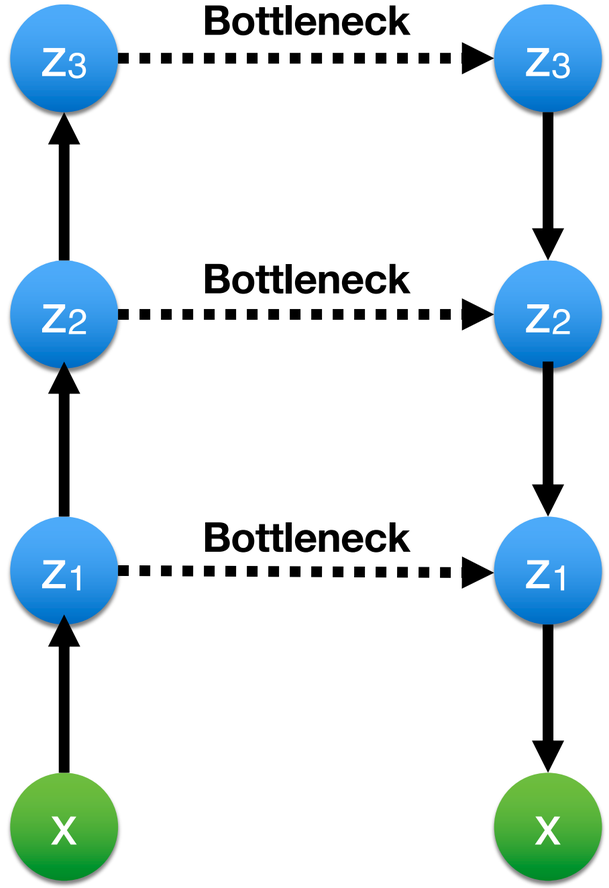
VAEs: challenges

- Optimization can be difficult for large models
- The ELBO enforces an **information bottleneck** (through its loss function) at the latent variables 'z', making VAE optimization prone to **bad local minima**.
- **Posterior collapse** is a dreaded bad local minimum where the latents do not transmit any information.

EXPLANATION

Posterior collapse, intuitively: if the decoder is powerful enough to model x on its own, the cheapest way to lower the loss is to make q(z|x) = p(z) (KL term = 0) and ignore z entirely. The latents then carry no information.

Common remedies: KL annealing / warm-up, “free bits”. Diffusion models sidestep the problem by **not learning the encoder at all**.



Inference model **q(z|x)**

Generative model **p(x,z)**

[Kingma and Welling, 2013]

Diffusion Models

OVERVIEW

Core idea: destroying an image with noise is easy and needs no learning; **undoing a tiny bit of noise** is a simple, learnable task. Chain many such tiny undo steps and you can turn pure noise into a realistic image.

Denoising Diffusion Models

Learning to generate by denoising

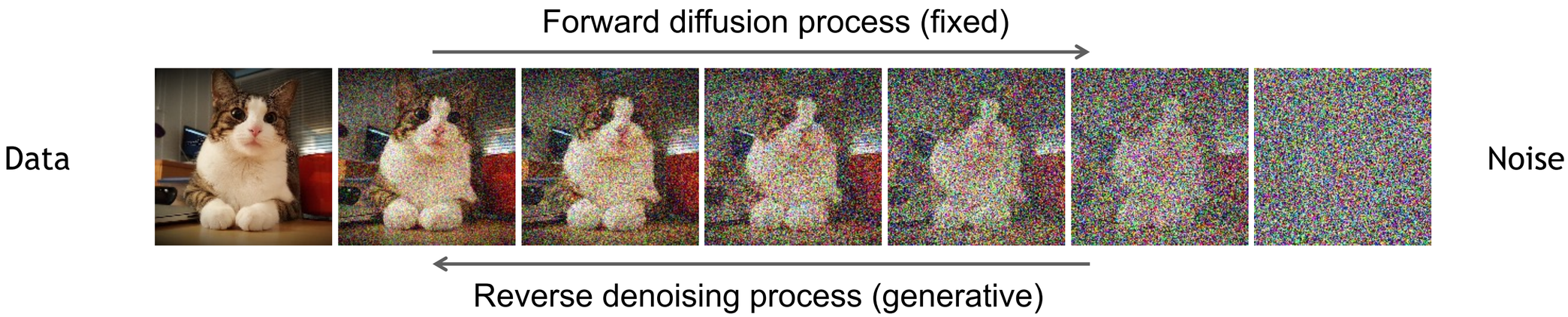
Denoising diffusion models consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising

EXPLANATION

Intuition: like a sculptor removing material bit by bit. A single giant leap from noise to image is too hard to learn, but each small denoising step is nearly Gaussian and easy to model.

Only the **reverse** process has learnable parameters.



Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015
Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020
Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021

Forward Diffusion Process

The formal definition of the forward process in T steps:

EXPLANATION

Each step slightly **shrinks** the signal (by $\sqrt{(1-\beta_t)}$) and **adds** a little Gaussian noise (variance β_t). In DDPM, T = 1000 and β_t rises linearly from 10^{-4} to 0.02. No learnable parameters.

Forward diffusion process (fixed)

Data



Noise

x_0 x_1 x_2 x_3 x_4 ... x_T

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t\mathbf{I}) \quad \Rightarrow \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}) \qquad \text{(joint)}$$

Similar to the inference model in hierarchical VAEs.

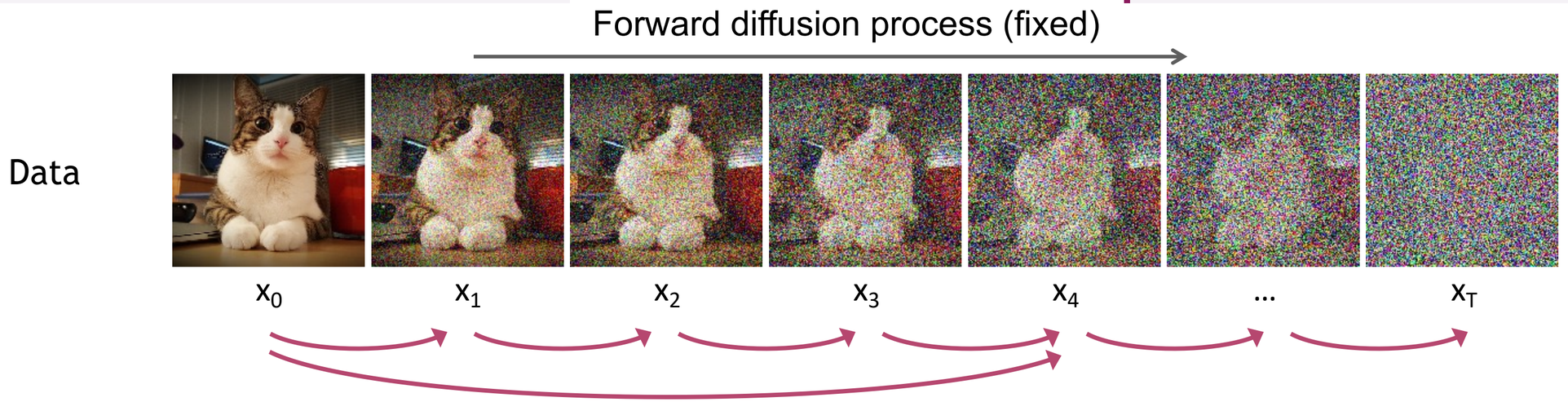
Diffusion Kernel

EXPLANATION

Gaussians compose: many small noise steps add up to **one** Gaussian step, so x_t can be drawn directly from x_0.

EXPLANATION

Why it matters: training is cheap — pick a random t, draw one ε, and form x_t in a single shot.



Define $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ ➡ $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}))$ (Diffusion Kernel)

For sampling: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

β_t values schedule (i.e., the noise schedule) is designed such that $\bar{\alpha}_T \to 0$ and $q(\mathbf{x}_T|\mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}))$

$\bar{\alpha}_t$ = fraction of the original signal variance left at step t: close to 1 (clean image) at the start, close to 0 (pure noise) at t = T.

Generative Learning by Denoising

Recall, that the diffusion parameters are designed such that $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}))$

Generation:

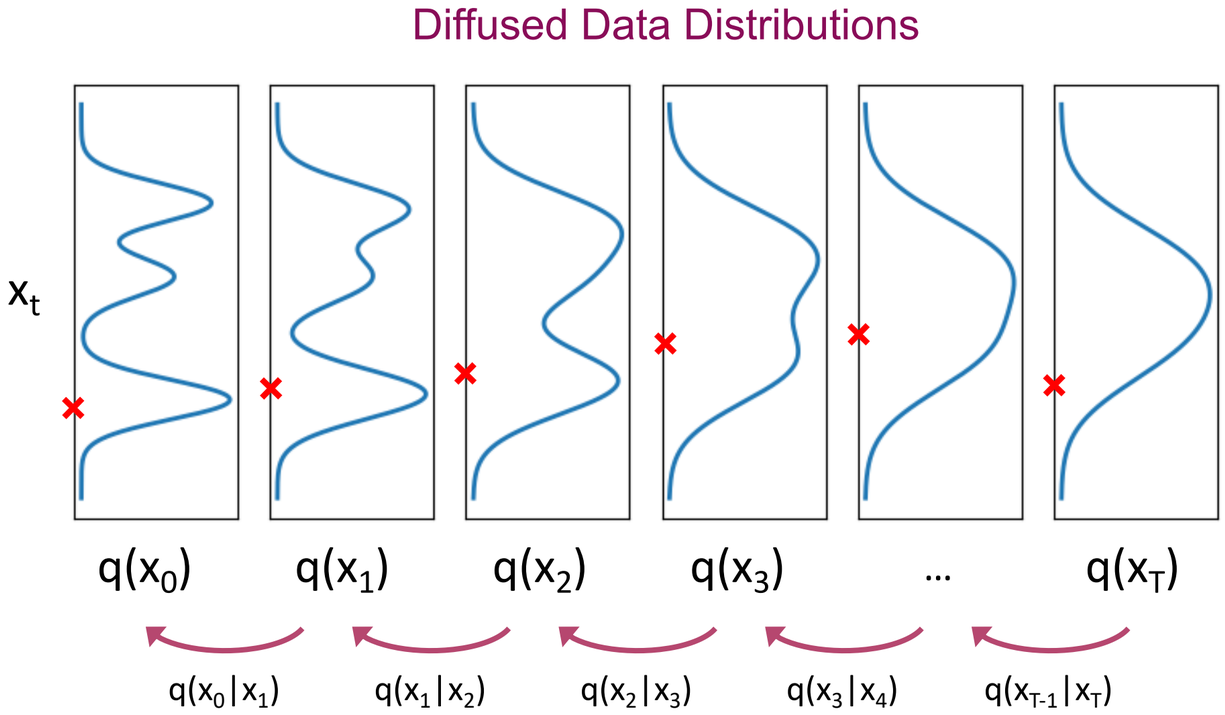
Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}$

True Denoising Dist.

> **EXPLANATION**
> **Why intractable?** It needs $q(x_{t-1})$, the distribution of *all* noisy images — unknown. But when each step adds only a little noise, the true reverse step is ≈ Gaussian, so the network just learns its **mean**.

In general, $q(\mathbf{x}_{t-1}|\mathbf{x}_t) \propto q(\mathbf{x}_{t-1})q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is intractable.



Can we approximate $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$? Yes, we can use a Gaussian distribution if β_t is small in each forward diffusion step.

Reverse Denoising Process

Formal definition of forward and reverse processes in T steps:

EXPLANATION

Start from pure noise x_T and apply the same learned step T times. One network $\mu_\theta(x_t, t)$ is **shared across all steps**; t is given as an input so it knows how much noise to expect. The variance σ_t^2 is fixed (e.g. β_t), not learned.

Reverse denoising process (generative)

Data



Noise

x_0 x_1 x_2 x_3 x_4 ... x_T

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \underbrace{\mu_\theta(\mathbf{x}_t, t)}, \sigma_t^2\mathbf{I})$$

Trainable network
(U-net, Denoising Autoencoder)

➡

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Similar to the generative model in hierarchical VAEs.

Learning Denoising Model

Variational upper bound

For training, we can form variational upper bound (negative ELBO) that is commonly used for training variational autoencoders:

$$\mathbb{E}_{q(\mathbf{x}_0)}\left[-\log p_\theta(\mathbf{x}_0)\right] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] =: L$$

Sohl-Dickstein et al. ICML 2015 and Ho et al. NeurIPS 2020 show that:

$$L = \mathbb{E}_q\left[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))}_{L_T} + \sum_{t>1}\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1))}_{L_0}\right]$$

EXPLANATION

L_T: no parameters; ≈ 0 by design.

L_{t-1}: make each learned reverse step match the **true** reverse step. That target is tractable because it is conditioned on the clean x_0.

L_0: final decoding to pixels.

where $q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)$ is the tractable posterior distribution:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1};\tilde{\mu}_t(\mathbf{x}_t,\mathbf{x}_0),\tilde{\beta}_t\mathbf{I}),$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t,\mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1-\beta_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

Parameterizing the Denoising Model

Since both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ are Normal distributions, the KL divergence has a simple form:

$$L_{t-1} = D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2\right] + C$$

Recall that $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon$. Ho et al. NeurIPS 2020 observe that:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$$

They propose to represent the mean of the denoising model using a *noise-prediction* network:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon_\theta(\mathbf{x}_t, t)\right)$$

With this parameterization

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0\sim q(\mathbf{x}_0),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon}_{\mathbf{x}_t}, t)||^2\right] + C$$

EXPLANATION

In words: rather than predicting the denoised mean directly, the network predicts **the noise ε that was added**. The loss becomes a (weighted) mean-squared error between true and predicted noise.

This is equivalent to **denoising score matching**: ε_θ points (up to scale) along $-\nabla \log q(x_t)$.

Training Objective Weighting

Trading likelihood for perceptual quality

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \Big[\underbrace{\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}}_{\lambda_t} ||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \epsilon, t)||^2 \Big]$$

The time dependent λ_t ensures that the training objective is weighted properly for the maximum data likelihood training.

However, this weight is often very large for small t's.

Ho et al. NeurIPS 2020 observe that simply setting $\lambda_t = 1$ improves sample quality. So, they propose to use:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1,T)} \Big[||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \epsilon}_{\mathbf{x}_t}, t)||^2 \Big]$$

EXPLANATION

Why drop λ_t? The likelihood-correct weights are huge for small t, so training would focus on removing almost-invisible noise. With $\lambda_t = 1$, the model spends more effort on the high-noise steps that decide global structure — better-looking samples, at the cost of slightly worse log-likelihood. The result is simple regression: add noise, predict it, minimise MSE.

Summary

Training and Sample Generation

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

EXPLANATION

Training: pick a real image, a random step t and random noise; build the noisy x_t in one shot; teach the network to predict the noise. Repeat. Every iteration trains one random step.

EXPLANATION

Sampling: start from pure noise; at each step subtract the predicted noise (red box = the learned mean) and add a little fresh noise $\sigma_t z$. After T steps, x_0 is a new image.

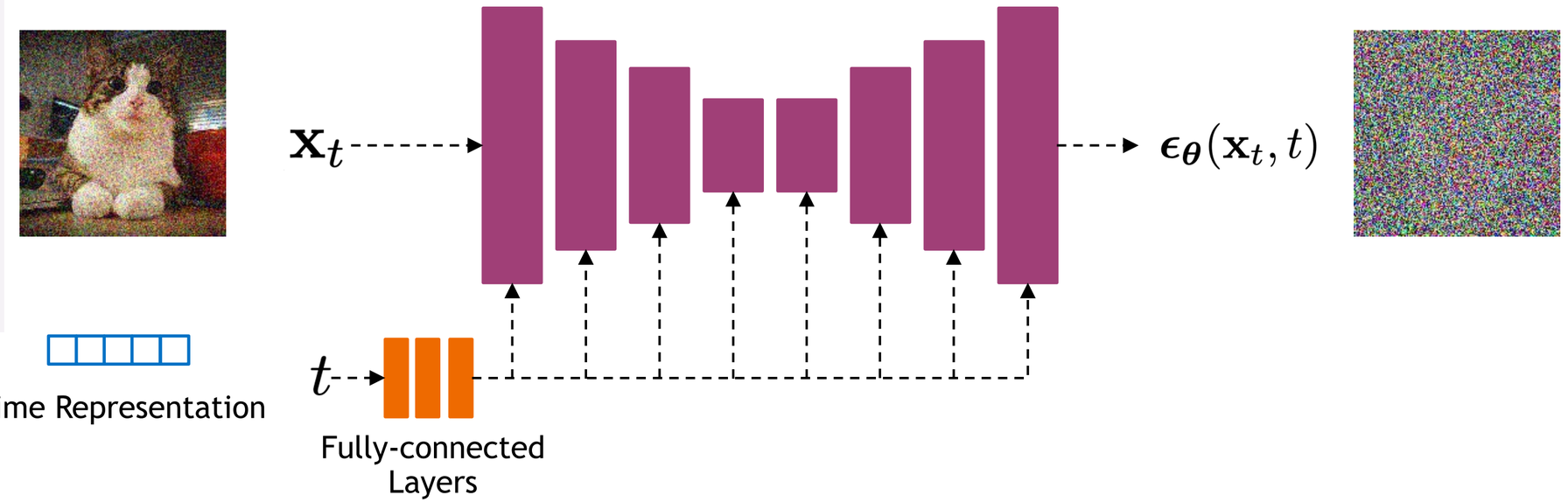
Implementation Considerations

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_\theta(\mathbf{x}_t, t)$

EXPLANATION

Why U-Net? Input and output have the same size (image in, noise map out). Skip connections keep fine detail; the bottleneck and self-attention capture global context.



EXPLANATION

Why a time input? The same weights handle every noise level. The embedding of t tells each block whether it is cleaning heavy noise (rough layout) or light noise (fine texture).

Time representation: sinusoidal positional embeddings or random Fourier features.

Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers. (see Dhariwal and Nichol NeurIPS 2021)

Connection to VAEs

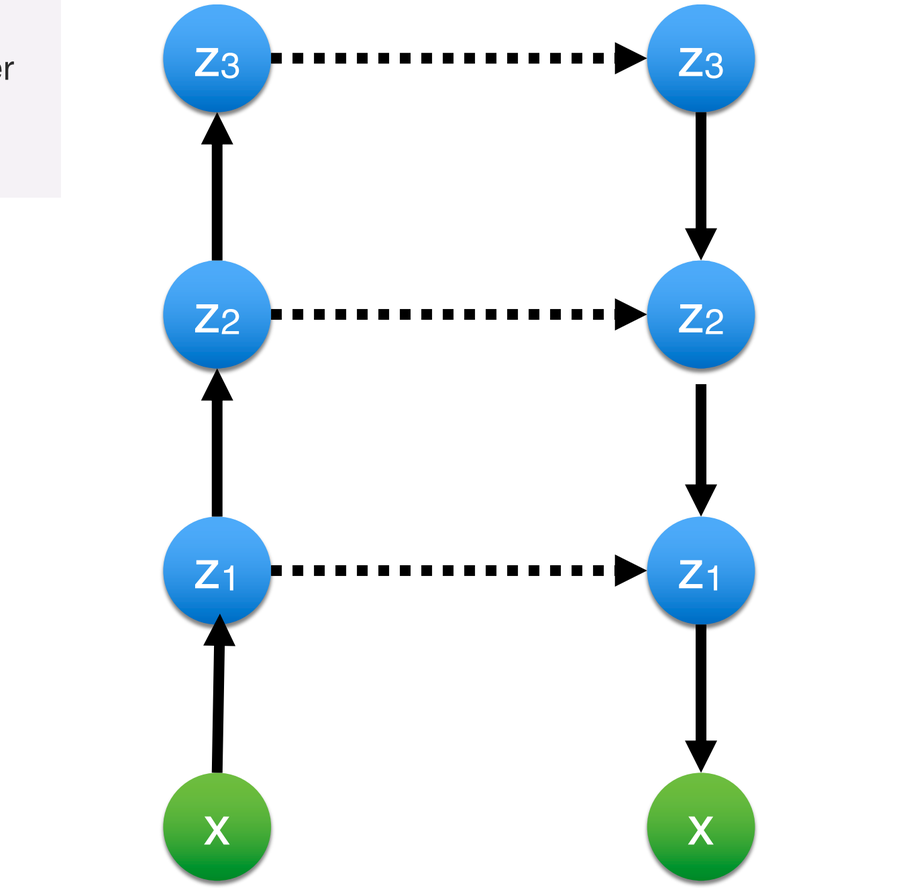
EXPLANATION

Takeaway: diffusion = a hierarchical VAE with the encoder frozen to “add noise” — no encoder to learn, no posterior collapse. Price: latents are as big as the image, so sampling is costly; **latent diffusion** (Stable Diffusion) diffuses in a compressed space.

Diffusion models can be considered as a special form of hierarchical VAEs.

However, in diffusion models:

- The inference model is fixed: easier to optimize
- The latent variables have the same dimension as the data.
- The ELBO is decomposed to each time step: fast to train
    - Can be made extremely deep (even infinitely deep)
- The model is trained with some reweighting of the ELBO.



Vahdat and Kautz, NVAE: A Deep Hierarchical Variational Autoencoder, NeurIPS 2020
Sønderby, et al.. Ladder variational autoencoders, NeurIPS 2016.

How to make sampling faster?

- One bottleneck of diffusion models is its slowness in sampling: need 10-1000+ steps to generate high quality samples
- Generative models need to be fast for practical use.

- One solution: distill diffusion models into models using just 4-8 sampling steps!
  - *Progressive distillation for fast sampling of diffusion models, Salimans & Ho, ICLR 2022*
  - *On Distillation of Guided Diffusion Models, Meng et al., CVPR 2023*

EXPLANATION

Why so slow? Each step is a full forward pass of a large U-Net, so 1000 steps = 1000 network evaluations per image (a GAN needs one).

Progressive distillation: a student learns to match two teacher steps in one; the student then becomes the teacher, halving the step count each round (e.g. 1024 → 512 → … → 4).

Other routes: deterministic samplers such as DDIM (tens of steps), fast ODE solvers (DPM-Solver), consistency models (1–2 steps), and running diffusion in a smaller latent space.

Case study: Imagen

OVERVIEW

How the ideas above scale to a real system: a frozen large **language model** reads the prompt, and a cascade of **pixel-space diffusion** models turns it into a high-resolution image.

Imagen: text-to-image diffusion models

By Google (imagen.research.google)

Input: text; Output: 1kx1k images

- An unprecedented degree of photorealism
    - SOTA automatic scores & human ratings
- A deep level of language understanding
- Extremely simple
    - no latent space, no quantization



A brain riding a rocketship heading towards the moon.

EXPLANATION

Pipeline: frozen T5-XXL text encoder → 64×64 base diffusion model → two super-resolution diffusion models (256^2, then 1024^2). **Key finding:** scaling the **text encoder** improved results more than scaling the image U-Net. “No latent space” = it diffuses directly in pixels, unlike Stable Diffusion.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

By Google (imagen.research.google)

EXPLANATION

What this prompt tests: several objects and attributes must be bound correctly — the dog has the backpack, wears both the sunglasses and the beach hat, and is riding the bike.

Getting all of them right in one image is evidence of **deep language understanding**, which Imagen attributes to its large pretrained text encoder.



A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.

EXPLANATION

Classifier-free guidance: the model is trained both with and without the text. At sampling time it extrapolates from the unconditional prediction toward the text-conditioned one, trading diversity for closer prompt adherence.

Imagen adds **dynamic thresholding** so very strong guidance doesn't saturate pixel colours.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

By Google (imagen.research.google)

EXPLANATION

Compositional generalization: a dragon fruit with arms, wearing a karate belt, in snow is unlikely to exist in any training image.

The model must combine concepts it learned separately — fruit texture, a martial-arts pose, snowfall — into one coherent scene.



A dragon fruit wearing karate belt in the snow.

EXPLANATION

How it was evaluated: automatic metrics (FID on COCO; CLIP score for text alignment) plus **DrawBench**, a set of challenging prompts (counting, colours, spatial relations, rare words) judged by human raters, where Imagen was preferred over DALL·E 2 and other models.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.